

Maulana Abul Kalam Azad University of Technology of West Bengal
Seacom Engineering College

ExamAI

AI-Powered Secure Examination Portal

Presented By:

Sumangal Kayal

B.Tech Final Year Project
Department of Information Technology

B.TECH CAPSTONE

EXAMINATION
PORTAL

AI PROCTORING

Index & Presentation Outline

- 01.** Project Overview & Introduction
- 02.** Problem Statement & Project Scope
- 03.** Core System Features (Tri-Role Panel)
- 04.** System Architecture & Flow
- 05.** AI Proctoring & Verification (face-api.js)
- 06.** Database Schema & API Design
- 07.** Technical Implementation Stack
- 08.** System Workflow & Process Steps
- 09.** Future Enhancements & Additions
- 10.** Academic Relevance & Final Summary

Introduction & Project Overview

Next-Gen Assessment Portal: ExamAI is a modern, full-stack online examination system designed to manage remote assessments securely.

Roles & Capabilities: Integrates three distinct functional roles - Student (examinee), Examiner (invigilator/creator), and Administrator (system audit/oversight).

Intelligent Monitoring: Protects academic integrity using client-side AI analysis to capture anomalies (window tab switches, facial absences, multiple users).

Autonomous Pipeline: Streamlines assessment creation, answer evaluation, risk log processing, and final dashboard analytics in a single environment.

EXAMAI HIGHLIGHTS

- MERN Stack + Client AI
- Tri-Role Access Boundaries
- Real-Time Incident Tracking
- Automated Evaluation Engine

Problem Statement & Necessity

Integrity Risks: Traditional remote tests lack direct oversight, leading to widespread cheating, tab switching, and unauthorized aids.

Scalability Limits: Manual remote invigilation requires a high ratio of human proctors, causing excessive costs and human oversight fatigue.

LMS Shortcomings: Standard Learning Management Systems do not include integrated facial authentication or active background behavioral tracking.

Unified Platform Need: Institutions need a cost-effective, real-time proctored test environment that evaluates students and tracks risk indexes automatically.

CORE TARGET PROBLEMS

- Unverified candidate identity
- Frequent browser tab switches
- Exorbitant cost of human invigilation
- Scattered audit logs & charts

Core System Features

Student Dashboard: Features Secure login, Face-ID validation, active exam panel, automatic countdown autosave, and instantaneous grade delivery.

Examiner Workspace: Provides interface to design exams, author custom questions, review submissions, and inspect real-time anomaly timelines with risk scoring.

AI Proctoring Module: Tracks tab switches, window minimizations, facial presence, and calculates composite candidate risk quotients.

ADMIN CONTROLS

- System-wide analytics dashboard
- Interactive audit log viewer
- Database model security controls
- Account creation & management

System Architecture & Flow

Client-Server Decoupling: React frontend communicates with Express backend through structured REST API endpoints.

Security Architecture: Stateless authentication via JSON Web Tokens (JWT) stored client-side; password encryption utilizing bcryptjs.

Database Management: Express communicates with MongoDB Atlas via Mongoose ODM for reliable schemas.

AI Pipeline: Face detection model runs fully on the client browser via face-api.js, saving server resources and bandwidth.

DATA FLOW STEPS

- 1. Auth & JWT generation
- 2. Client-side face descriptor check
- 3. Exam delivery & active timers
- 4. Event logs pushed to server DB
- 5. Live examiner dashboard updates

AI Proctoring & Face-ID Verification

Face Descriptors: Extracts a 128-dimensional mathematical vector (descriptor) representing distinct facial features via deep learning models.

Verification Process: During verification, face-api.js matches live webcam descriptors against the registered vector using Euclidean distance.

Tab Switch Logging: Intercepts browser Page Visibility API events to record when a candidate leaves the examination window.

Risk Scoring: Computes a cumulative danger metric based on incident logs. Examiners get immediate telemetry on candidate violations.

MONITORED VIOLATIONS

- Tab changes & focus losses
- No face detected in webcam
- Identity validation failures
- Audio/visual threshold warnings

Database Models & API Modules

Mongoose Schema Models: Utilizes structured models for Users (roles, hashes, face descriptors), Exams, Submissions, and Incidents.

Auth Modules: Supports registration, JWT token generation, and secure payload decryption on middleware routes.

Exam & Submission APIs: Facilitates CRUD operations on questions, real-time response commits, and automated score compiling on exam termination.

Incident Logging APIs: Stores timelines of violations and fetches analytics datasets for examiners.

CORE API ENDPOINTS

- POST /api/auth/login
- POST /api/auth/register-face
- GET/POST /api/exams
- POST /api/submissions
- POST /api/incidents/log

Technical Stack Details

Frontend (React 19 & Vite 8): Responsive UI utilizing modern react hooks, React Router 7 for secure routing boundaries, and vanilla CSS for styling.

Backend (Node.js & Express 5): Robust, async API layer, CORS support, secure file processing via Multer, and routing middlewares.

Database (MongoDB Atlas): Flexible document-based storage, high availability indexing, and cloud replication.

Telemetry & AI: Recharts for telemetry dashboards, Lucide React icons, and local face-api.js web models.

KEY LIBRARIES

- face-api.js (Local ML)
- Recharts (Visual analytics)
- bcryptjs (Password hash)
- jsonwebtoken (Auth token)

Future Scope & Enhancements

Live Audio Monitoring: Integration of noise capture algorithms to flag verbal discussions and background help anomalies.

Dedicated Lockdown Browser: Building an OS-specific wrapper application to disable screenshots, secondary displays, and shortcut commands.

Server-Side AI Audits: Moving heavy visual models to backend nodes for periodic feed reviews.

Automated Notification System: Incorporating SMS or email auto-warnings when a student violates proctor bounds multiple times.

VALUE ADDITIONS

- Multi-face detection models
- Automatic exam certificate generation
- Live invigilator audio/video calls
- Predictive ML cheating models

Academic Relevance & Summary

MERN Demonstration: Showcases professional full-stack Javascript mastery, data schema optimizations, and security controls.

Browser ML Implementation: Illustrates the capability of executing AI models directly in-browser for zero-cost client processing.

Enterprise Readiness: Maintains strict separation of concerns, JWT stateless routes, and interactive admin dashboards.

Conclusion: ExamAI resolves core online testing vulnerabilities, proving that smart software solutions can effectively secure distant learning.

PRESENTATION SUMMARY

- Secure MERN Architecture
- Real-Time Client-Side AI
- Optimized DB schemas
- Flexible Future-Ready Design

THANK YOU!

Questions & Discussions are Welcome.

Project: ExamAI - Secure Examination Portal

Student Name: Sumangal Kayal | B.Tech CSE Final Year

College Name: [Your College/University Name]